

Orbiter的去中心化之路 · SPV

深度、持久、探索 学习

去中心化的核心问题是什么？

在没有**单一中央实体**的情况下，如何让系统能够**自主**地完成其功能，保持**安全性**和**可信性**，并且不依赖于**单点控制**。

什么是SPV? 为什么需要SPV?

- "Simplified Payment Verification" (简单支付验证) 最初是在比特币的白皮书中提出的。它通过利用默克尔证明的特性, 允许交易接收者证明发送者对所提供的支付源资金拥有控制权, 而无需下载完整的区块链。
- **SPV**的作用是在资源有限的环境下, 实现快速、轻量级的区块链验证。

SPV的三大关键指标是什么？

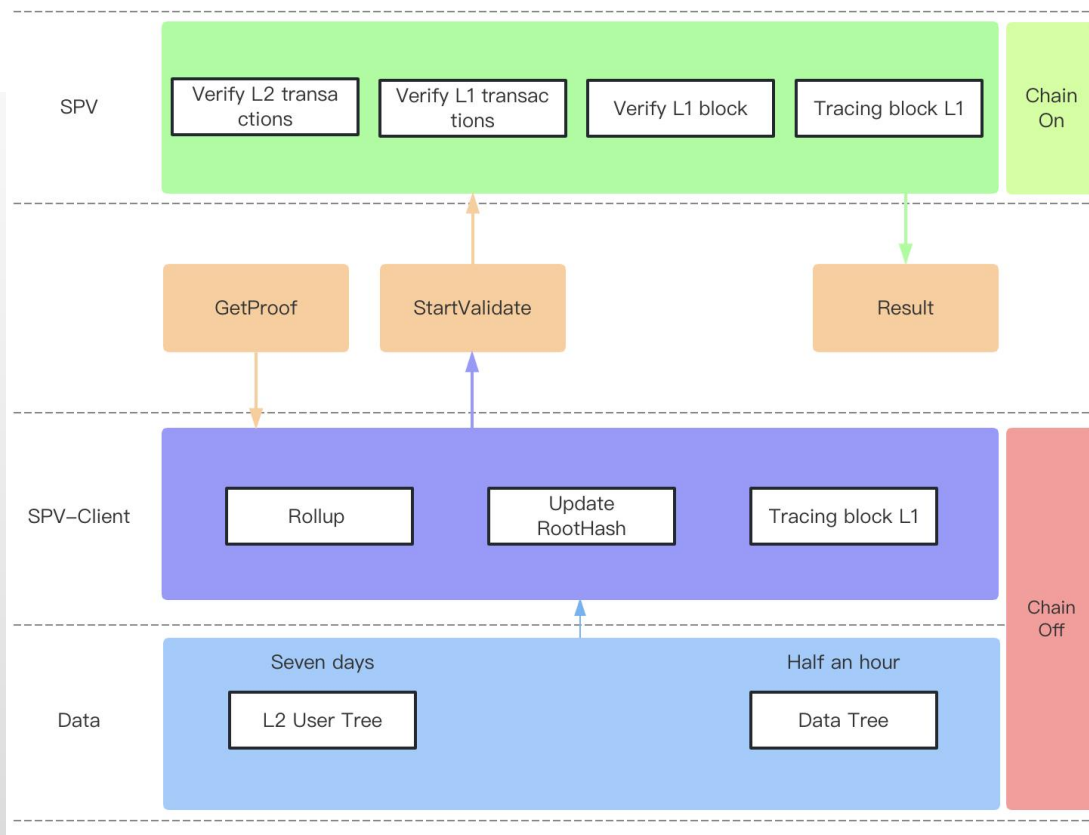
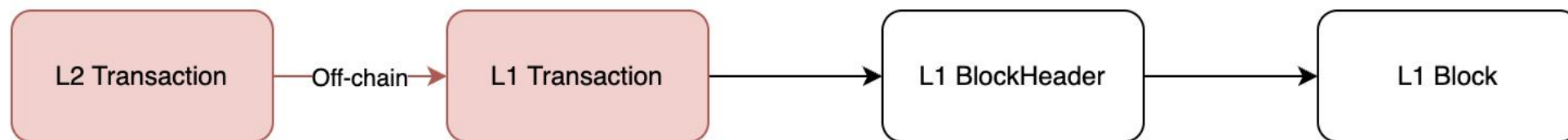
1. 安全性（确保交易的有效性和区块链数据的完整性）
2. 可信性（用户能够对所接收的区块链数据和交易数据产生足够的信任）
3. 可用性（在不同环境和使用场景下的适用性和可行性）

Orbiter需要SPV达到什么目的？

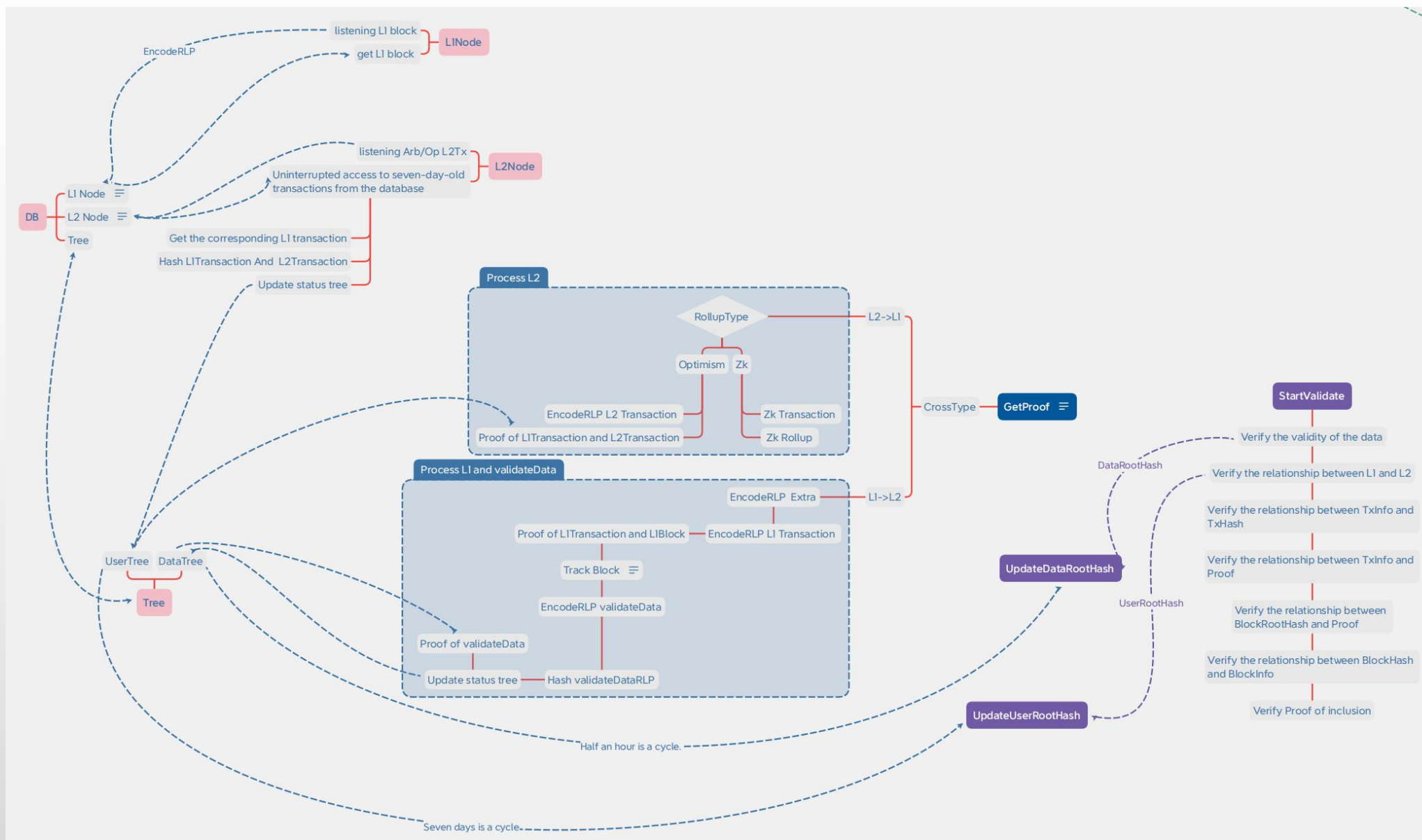
证明Orbiter cross-rollup Tx的存在性和合理性。存在性是指Source交易和Destination交易都可以在L1上证明它们确实发生在相应的L2上，合理性是指能够在SrcTx中证明用户的意图，并确保DstTx中做市商的支付结果符合特定规则。

—— 《Orbiter Yellow Paper》

SPV-原始版本



SPV-原始版本



SPV-原始版本

SPV-原始版本的问题

1. 安全性
 1. 部分证明数据来源于链下构造（证明L2 Tx包含于L1 Tx中、证明256个区块之外的区块）
 2. 未证明L2 Tx的有效性，即我们只证明了它的存在性
2. 可信性
 1. 部分数据来源于链下构造，形成中心化趋势，用户不能够直接信任SPV证明的数据
3. 可用性
 1. Gas消耗高，即使优化后，也达到大约300w gas

SPV-原始版本

SPV-原始版本的解决尝试

1. 针对交易有效性问题，我们在链下使用默克尔树进行存储交易的数据，在链上使用默克尔证明
2. 针对数据安全问题，我们也在链下使用默克尔树对链下**SVP**客户端的数据进行存储，在链上使用默克尔证明
3. ...
4. 最终导致链下需要维护的数据较多，且只能由我们或者可信的人提交树的哈希至链上以用于默克尔证明

优化

思路？

在解决中心化问题的同时，解决安全性与可用性问题

优化

如何解决中心化问题？

产生中心化问题的主要因素是部分证明数据的数据需要在链下维护的，且维护和提交这部分数据的人只能是我们或者可信的人。

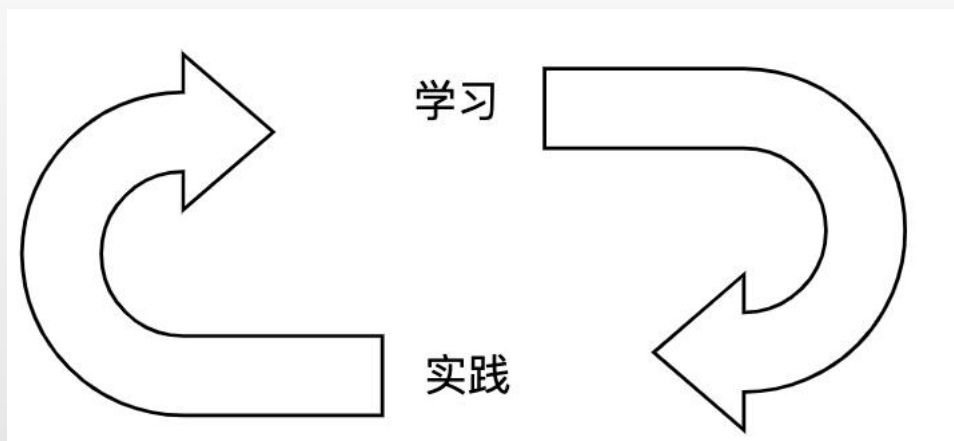
优化

去中心化过程中产生的新问题?

1. 证明L2 Tx的有效性
2. 证明L2 Tx包含于L1 Tx中
3. 证明L1 256区块之外的区块
4. 解决Gas消耗高的问题
- ...

优化

如何解决去中心化过程中产生的新问题？



优化

如何学习？

1. 深度学习-研究要求将精力集中在一个特定的领域或问题上
2. 持久学习-研究是一个漫长而充满挑战的过程，不轻易放弃
3. 探索学习-研究并不是僵化地坚守某一领域，而是通过创新和探索，不断推动该领域的边界

优化

如何实践？

跑测试

优化

如何快速入门新领域

1. 了解新领域的基本概念
2. 了解新领域的实现产品
3. 查找新领域的学习资料
4. 实践新领域的示例
5. 跟踪新领域的社区与发展

1. 了解zk的基本概念
2. 了解zk形式之一的工作原理和特点，包括生成证明、验证证明的精简描述
3. 查找可靠的学习资料，例如教程、博客文章、学术论文
4. 实践一些zk的示例，使用zk形式之一生成和验证简单的证明，以加深对其工作原理的理解
5. 参与zk社区、交流、跟踪最新研究和发展

优化

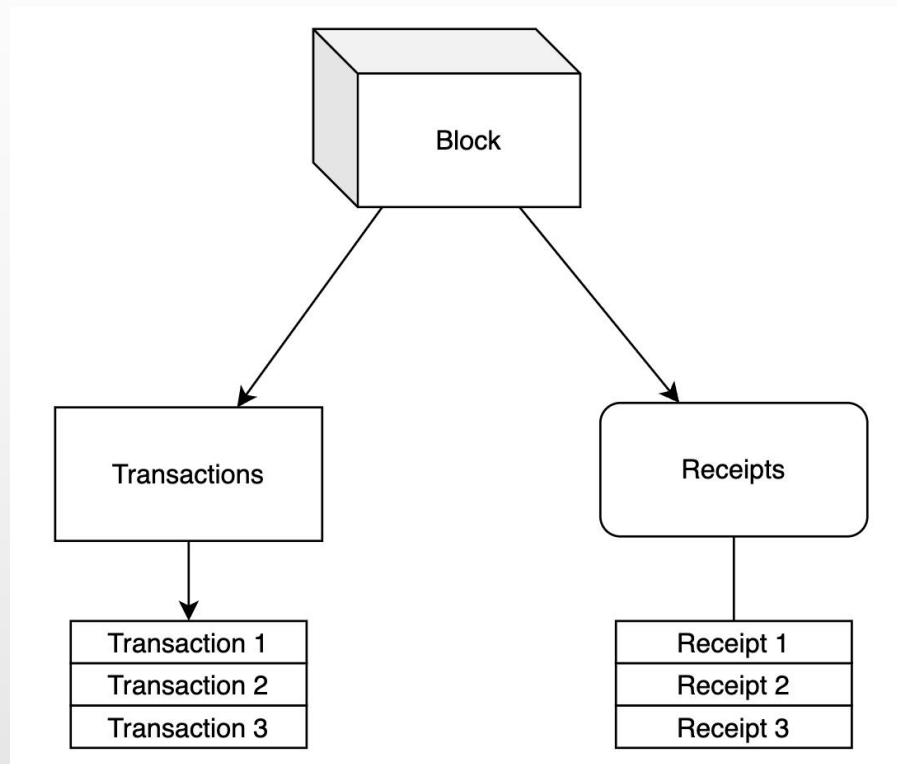
1. 证明L2 Tx的有效性:

由于同一个区块内收据树中的键和交易树中的键是一一对应的，所以具有相同键的值代表同一交易的对应数据。且交易的收据中**Status**字段标志着该交易的事务状态。

那么可以使用L2 Tx的receipt和L2 Tx所在的Block的**ReceiptsRoot**结合MPT进行证明。

2. 证明L2 Tx包含于L1 Tx的Input中:

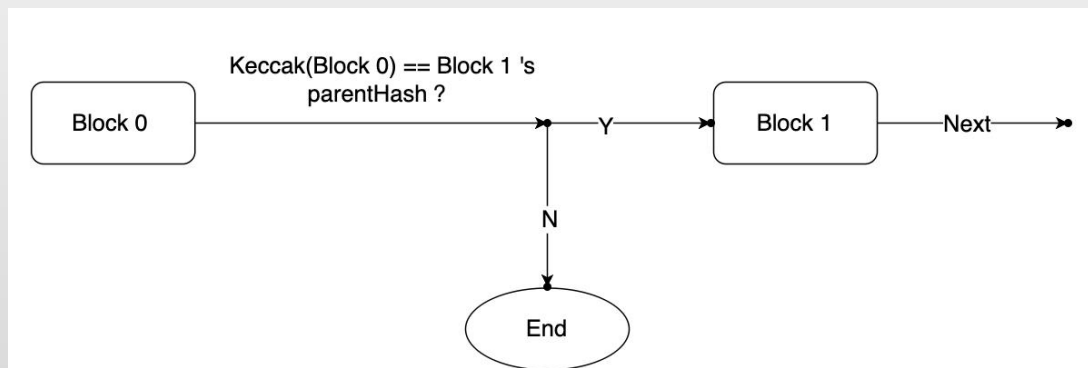
在基于arbitrum、optimism的定序器代码库上，编写出适用于各自rollup的编解码工具，并加入证明



优化

3. 证明256区块之外的区块

1. L2: 基于L2机制, 可以肯定的是, **Layer 1**总是有相应的状态合约来存储特定的状态。因此, 可信区块来源于官方rollup实现提供的链上合约, 例如Arbitrum的Outbox合约、Optimism的L2OutputOracle合约、zkSync的Storage合约等。
2. L1: 为了减少链下计算的时间成本并增强链上可操作性, 我们有一个L1StorageBlockHash合约。理想情况下, 该合约每小时记录L1的最新区块哈希, 并以bytes32->uint256映射的形式存储。合约没有权限, 任何人都可以更新区块哈希。目前, 它由我们的官方团队维护, 但有计划激励其他人参与更新状态。由于这些块哈希值是使用blockhash()直接从合约中读取的, 因此可以认为它们是有效的。
3. 因此, 我们可以从目标开始区块计算前一个块的哈希值是否等于下一个块的父哈希值, 不断迭代直到到达可信区块 (可信区块 $\geq$ 目标开始区块)



优化

4. 解决Gas消耗高的问题

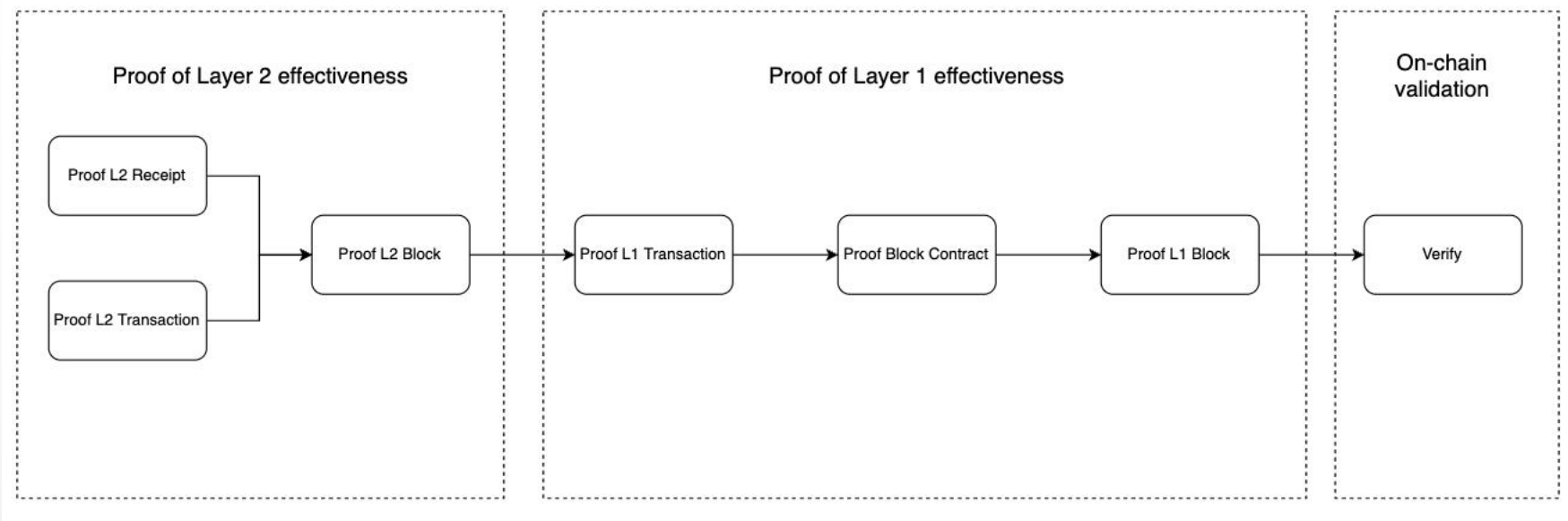
使用halo2结合Plonk实现递归证明

SPV-ZKP

成果

1. 低gas:由于我们在 Halo2 中使用 PLONK 进行递归证明组合，目前链上证明的 Gas 消耗平均约为 35w Gas（之前为 $\geq 300w$ gas）
2. 安全:无论是Layer 2还是Layer 1的有效性证明，最终都会回到区块证明，而区块证明不可避免地链上数据挂钩。只有数据正确才能通过区块证明，并在 zk-SNARK 证明中验证计算的最终有效性。
3. 通用:我们支持以太坊、Arbitrum、Optimism 和 zkSync Era 的交易有效性证明。除此之外，我们还提供历史区块有效性证明、合约状态有效性证明和账户状态证明服务。未来，我们计划将支持扩展到更多的 Rollup 网络。此外，我们的目标是允许第三方服务使用我们的服务。

SPV-ZKP

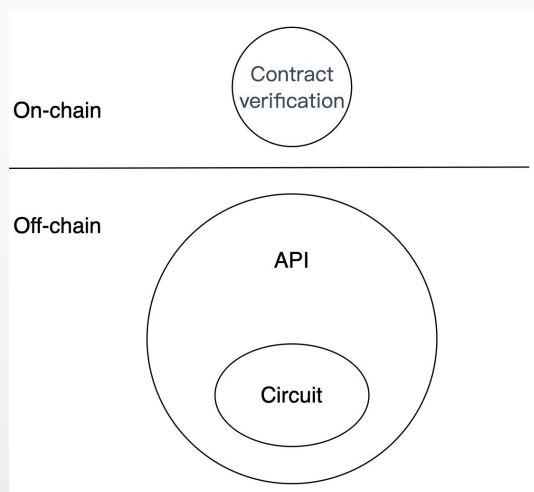


整个证明过程分为七个步骤（指证明过程中要采取的最长步骤）。

1. L2 Receipt有效性证明
2. L2 Transaction有效性证明
3. L2 BlockHeader有效性证明
4. L1 Transaction有效性证明
5. L1 Block Contract状态有效性证明
6. L1Transaction有效性证明
7. 链上合约验证

其中，步骤1、2、3称为第2层有效性证明，步骤4、5、6称为第1层有效性证明，步骤7为链上合约验证。因此，完整的有效性证明由Layer 2 有效性证明+ L1 层有效性证明组成。

SPV-ZKP



1. **API:** 一组专为SPV服务设计的链下RPC客户端。从外部来看，它们的功能是接收交易哈希和其他信息并生成 **zk-SNARK** 有效性证明。在内部，它们负责构建电路的输入数据并持久保存 **zk-SNARK** 证明。
2. **电路:** 基于 Halo 2 证明系统的 ZK 电路和库。
3. **合约验证:** 用于验证 **zk-SNARK** 有效性证明的链上验证合约。

SPV-ZKP

优化点

1. 时效性：我们不对链上数据进行实时 zk-SNARK 证明。相反，我们在发起需求时生成 zk-SNARK 证明。因此，及时性对我们来说至关重要。目前，我们的时效性受到两个因素的影响。首先，各种 Rollup 在第 1 层更新其相应状态合约的频率。其次，链下执行 zk 计算的复杂性，其中时间成本随着复杂性的增加而增加。目前，链下zk计算和证明生成所需时间为2至4小时（测试环境：128GB RAM，64核）。当然，我们正在不断努力优化电路生成和证明生成的时间。我们正在寻求两种方法：优化电路本身和探索 GPU 等硬件加速。
2. Gas消耗：目前，我们在Halo2中使用PLONK进行递归证明组合。为了减少链上证明的 Gas 消耗，我们正在探索使用 FFLONK 在 Halo2 中进行递归证明组合。

SPV-ZKP

市场机会探索

虽然我们目前支持交易有效性证明、合约状态存储证明、账户状态证明以及区块状态证明，但是仍在探索更多的适用场景

没了